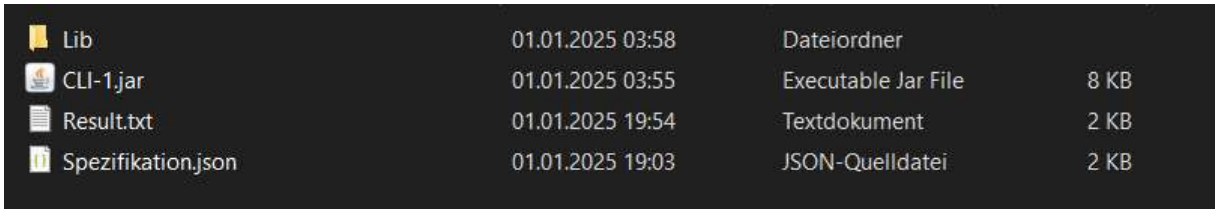
Daniel Brenner, Dinh Xuan Truong Ung, Amine Rekik, Roman Pauli

Aufgabe 6: Architecture Validation

Annahmen und Voraussetzungen:



CLI-1.jar:

- Kann beliebig benannt werden
- Stellt den Haupteintritt in das Softwaresystem dar

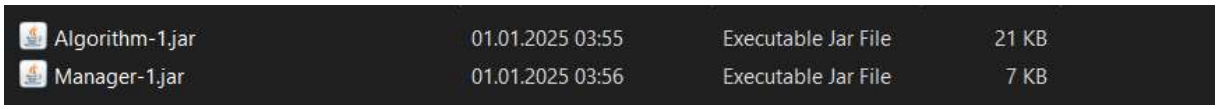
Result.txt:

- Wird erstellt nach einem Durchlauf (muss also nicht manuell erstellt werden)

Spezifikation.json:

- Kann beliebig benannt werden
- Definierte Architektur für das System
- Tiefere Stufe muss immer anderen ‚type' haben als höhere

```
{
  "defaultRule": "FORBIDDEN",
  "rules": [
    {
      "from": "com.reg.core",
      "to": "Address-Manager.jar",
      "type": "ALLOWED",
      "except":[
        {
          "from": "com.reg.core.impl",
          "to": "Address-Manager.jar",
          "type": "FORBIDDEN"
        }
      ]
    },{
      "from": "com.reg.utils.Foo",
      "to": "com.xdb.adapter.Bar",
      "type": "ALLOWED"
    }
  ]
}
```



Lib directory:

- Enthält Abhängigkeiten; kann ebenfalls einen beliebigen Namen tragen

Allgemein können beliebig viele so aufgebaute Analysen in das Verzeichnis src/main/scala/Analysis hinzugefügt werden.

Konkrete Nutzung:

Analyse:

Das Objekt task6 ist der Startpunkt. Um eine Analyse zu starten, muss man einfach den Run-Befehl einstellen und ausführen:

Vorhandenes Beispiel:

```
filePath=src/main/scala/Analysis/Analyse2 isLibrary=false
```

filePath: Path zum Analyse-Directory, welches, wie im vorherigen Teil definiert, aufgebaut sein muss

isLibrary: Gibt an, ob es sich nur um eine Library handelt oder ein Programm mit einem main-Eintritt

Das Ergebnis wird sowohl in die Ausgabe ausgegeben als auch in einer Result.txt gespeichert. Dabei werden zuerst Methoden aufgelistet, welche die definierte Architektur verletzten, aufgelistet und danach die Felder.
Der Aufbau ist wie folgt:

From: Gibt in welcher Methode der Verstoß ist.

To: Gibt an welche Methode aufgerufen wird.

Rule: Gibt an welche Rule verletzt wird

Nummer: Zum Schluss eine Zahl, die angibt, wie häufig dieser Verstoß gefunden wurde (falls mehrfach etwas gecallt wird).

```
Method: From 'UI.CLI.ListCli', To: 'org.Example.Algorithm.Util.ResultManager' Rule: From 'CLI-1.jar, To: 'org.Example.Algorithm.Util' 4
Method: From 'UI.CLI.GraphCli', To: 'org.Example.Algorithm.Util.ResultManager' Rule: From 'CLI-1.jar, To: 'org.Example.Algorithm.Util' 4
SPECIAL: From 'UI.CLI.OverviewCli', To: 'UI.CLI.Cli' Rule: From 'UI.CLI.OverviewCli, To: 'UI.CLI.Cli' 1
Method: From 'UI.CLI.OverviewCli', To: 'UI.CLI.Cli' Rule: From 'UI.CLI.OverviewCli, To: 'UI.CLI.Cli' 2
SPECIAL: From 'org.Example.Algorithm.Graph.Algorithm.DFS', To: 'org.Example.Manager.Creation.Graph.Node' Rule: From 'org.Example.Algorithm.Graph.Algorithm, To: 'org.Example.Manager.Creat
Method: From 'org.Example.Algorithm.Graph.Algorithm.DFS', To: 'org.Example.Manager.Creation.Graph.Node' Rule: From 'org.Example.Algorithm.Graph.Algorithm, To: 'org.Example.Manager.Creati
SPECIAL: From 'org.Example.Algorithm.Graph.Algorithm.BFS', To: 'org.Example.Manager.Creation.Graph.Node' Rule: From 'org.Example.Algorithm.Graph.Algorithm, To: 'org.Example.Manager.Creat
Method: From 'org.Example.Algorithm.Graph.Algorithm.BFS', To: 'org.Example.Manager.Creation.Graph.Node' Rule: From 'org.Example.Algorithm.Graph.Algorithm, To: 'org.Example.Manager.Creati

Only important if a field is is not set.1
Field : From 'org.Example.Algorithm.Graph.Algorithm.DFS.Nodes', To: 'org.Example.Manager.Creation.Graph.Node' Rule: From 'org.Example.Algorithm.Graph.Algorithm, To: 'org.Example.Manager.
Field : From 'org.Example.Algorithm.Graph.Algorithm.BFS.Nodes', To: 'org.Example.Manager.Creation.Graph.Node' Rule: From 'org.Example.Algorithm.Graph.Algorithm, To: 'org.Example.Manager.
```

Konfigurationsmöglichkeiten:

Nichts